Ollama + ROCm on a System76 Pangolin15

Running large language models on an "unsupported" AMD Radeon 780M iGPU under Pop!_OS 24.04

> *A weekend project disguised as a runbook. Or maybe the other way around.*

TL;DR

If you have a System76 Pangolin15 (AMD Ryzen 9 8945HS + Radeon 780M, gfx1103), Pop!_OS 24.04, and 96GB of DDR5 RAM, this guide gets you from "Ollama runs on CPU only" to "Ollama runs 14B-parameter models on your iGPU at 70% of theoretical hardware peak" in about 90 minutes.

Final result on this build:

- `llama3.2:3b` Q4 — **26 tok/s** (instant)
- `qwen2.5:7b` Q4 — ~18 tok/s (snappy chat)
- `qwen2.5:14b` Q4 — **6.2 tok/s** (usable for non-real-time tasks)
- Two models loaded simultaneously, 100% GPU offload, on a laptop iGPU AMD officially does not support

The history-teacher backstory

Class, gather round. Today's lesson is about how marketing departments lie to you and how Linux makes them honest again.

The setup. AMD's Ryzen 9 8945HS ships with an integrated GPU called the Radeon 780M. AMD's marketing labels this "Ryzen AI." This is misleading on two counts. First, the 780M is not an NPU — it's a graphics chip that happens to do compute. Second, AMD's official compute software stack (ROCm) does not include the math libraries for the 780M's specific silicon variant (codename **gfx1103**). The stack supports gfx1100, gfx1101, and gfx1102 — three close cousins — but not gfx1103.

The hack. It turns out gfx1102 and gfx1103 are binary-compatible enough that the math kernels compiled for one will run on the other. So we tell ROCm a small lie via the `HSA_OVERRIDE_GFX_VERSION=11.0.2` environment variable, which translates to "use the gfx1102 library." ROCm shrugs, loads the library, and the hardware happily executes it.

The memory trick. AMD iGPUs share system RAM with the CPU. Windows typically reserves a fixed chunk (the UMA Frame Buffer) in BIOS. On Linux there's a smarter mechanism — **GTT memory** (Graphics Translation Table) — where the GPU borrows pages from system RAM on

demand. We allocate 32GB of GTT via kernel boot parameters, leaving plenty of room for big models without permanently stealing RAM from anything else.

The Ollama gate. Recent Ollama builds (0.30+) deliberately ignore integrated GPUs unless you opt in with `OLLAMA_IGPU_ENABLE=1`. This is policy, not a technical limitation. Flip the switch.

That's the entire trick. Five environment variables and three kernel boot parameters between you and local LLMs on a laptop iGPU.

Prerequisites

- **Hardware**: System76 Pangolin15 (`AHP938` firmware family) with AMD Ryzen 9 8945HS + Radeon 780M
- **OS**: Pop!_OS 24.04, kernel 7.0.x or newer
- **RAM**: 32GB minimum (96GB recommended for headroom + bigger models)
- **Disk**: ~20GB free for ROCm packages and a starter set of models
- **Bootloader**: systemd-boot (Pop!_OS default — managed by `kernelstub`)

Verify your hardware before starting:

```
lscpu | grep "Model name"
# Expect: AMD Ryzen 9 8945HS w/ Radeon 780M Graphics
lspci | grep -i vga
# Expect: a line mentioning AMD Phoenix or Radeon 780M
```

Step 1 — System prep

```
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y curl wget gnupg lsb-release radeontop
```

`radeontop` gives you a live view of GPU utilization later. Trust me, you'll want it.

Step 2 — Allocate GTT memory via kernel parameters

The Pangolin15 BIOS hides the UMA Frame Buffer Size setting (System76 deliberately ships it locked at Auto/2GB). This is actually fine, because Linux's GTT mechanism is more flexible than BIOS-time UMA allocation anyway.

We're going to allocate **32GB of GTT** — enough for any model that fits the 780M's bandwidth ceiling, with headroom to spare.

2a — Verify your bootloader

```
sudo kernelstub --print-config
```

If this runs cleanly and shows "Kernel Boot Options", you're on systemd-boot via `kernelstub`. Continue.

If it errors out, you're probably on a non-standard setup and should consult Pop!_OS bootloader docs before proceeding.

2b — Add GTT parameters

```
sudo kernelstub --add-options "amdgpu.gttsize=32768 ttm.pages_limit=8388608 ttm.page
```

What each parameter does:

Parameter	Effect
`amdgpu.gttsize=32768`	Tells amdgpu the GTT pool can grow to 32GB (value in MB). Deprecated but still honored.
`ttm.pages_limit=8388608`	TTM memory manager: max pages it can hand to the GPU. Math: `32 GB ÷ 4 KB per page = 8,388,608`.
`ttm.page_pool_size=8388608`	Cached page pool size. Same value as above.

2c — Verify and reboot

```
sudo kernelstub --print-config
# Confirm the three new options appear in "Kernel Boot Options"
sudo reboot
```

2d — Confirm after reboot

```
cat /proc/cmdline
# Should show all three parameters

cat /sys/module/ttm/parameters/pages_limit
# Should print: 8388608

sudo dmesg | grep -iE "amdgpu.*(memory|gtt)" | head -5
# Should show a line like: "32768M of GTT memory ready."
```

If `lsmod | grep ttm` shows `amdttm` instead of `ttm`, swap the parameter prefix to `amdttm.pages_limit` and `amdttm.page_pool_size` in step 2b and redo. Pop!_OS 24.04 on kernel 7.0.x uses plain `ttm`.

Step 3 — Confirm group membership

ROCm talks to the GPU through `/dev/kfd` and `/dev/dri/*`, which are owned by the `render` and `video` groups.

```
groups
```

You should see both `render` and `video` in the output. On Pop!_OS 24.04 this is usually pre-configured. If either is missing:

```
sudo usermod -aG render,video $LOGNAME
sudo reboot
```

Step 4 — Install ROCm 6.2 (userspace only, no DKMS)

Critical: Pop!_OS ships its own `amdgpu` kernel module in the System76 kernel. Do **not** install `amdgpu-dkms` or `rocm-dkms` — they will fight the System76 kernel module and break your boot.

4a — Add the AMD ROCm repository

```
sudo mkdir -p /etc/apt/keyrings

wget https://repo.radeon.com/rocm/rocm.gpg.key -O - | \
  gpg --dearmor | sudo tee /etc/apt/keyrings/rocm.gpg > /dev/null

echo "deb [arch=amd64 signed-by=/etc/apt/keyrings/rocm.gpg] https://repo.radeon.com/r
  sudo tee /etc/apt/sources.list.d/rocm.list

sudo tee /etc/apt/preferences.d/rocm-pin-600 > /dev/null << 'EOF'
Package: *
Pin: release o=repo.radeon.com
Pin-Priority: 600
EOF

sudo apt update
```

The pin-priority file tells apt "prefer ROCm-repo versions over anything else" — prevents conflicting versions from Ubuntu's main repos.

4b — Install ROCm userspace

```
sudo apt install -y rocm-hip-libraries rocm-libs rocminfo rocm-smi
```

This is a ~3-4GB download. Make a sandwich.

4c — Verify ROCm sees the 780M

```
/opt/rocm/bin/rocminfo | grep -E "Marketing Name|gfx|Device Type"
```

Expected output (the GPU agent is what matters):

```
  Marketing Name:          AMD Ryzen 9 8945HS w/ Radeon 780M Graphics
  Device Type:             CPU
  Name:                    gfx1103
  Marketing Name:          AMD Radeon 780M Graphics
  Device Type:             GPU
      Name:                    amdgcn-amd-amdhsa--gfx1103
```

If you see `gfx1103` with `Device Type: GPU`, ROCm is talking to your hardware. This is the hardest milestone.

4d — Confirm the rocblas Tensile libraries

```
ls /opt/rocm/lib/rocblas/library/ | grep -E "gfx110[0-9]"
```

You should see files for `gfx1100`, `gfx1101`, `gfx1102`. **You will NOT see** `gfx1103` — that's the entire reason we need the override.

Step 5 — Install Ollama (skip if already present)

```
ollama --version
```

If this errors out, install Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Confirm:

```
ollama --version
systemctl status ollama --no-pager
```

Active and running. Note that the default install creates an `ollama` system user/group and a systemd service.

Step 6 — Configure Ollama for the 780M

Open the systemd drop-in editor:

```
sudo systemctl edit ollama
```

Between the two marker comments, add:

```
[Service]
Environment="HSA_OVERRIDE_GFX_VERSION=11.0.2"
Environment="OLLAMA_IGPU_ENABLE=1"
Environment="OLLAMA_FLASH_ATTENTION=1"
Environment="OLLAMA_KEEP_ALIVE=15m"
```

What each does:

Variable	Purpose
`HSA_OVERRIDE_GFX_VERSION=11.0.2`	The lie that makes gfx1103 use the gfx1102 Tensile library
`OLLAMA_IGPU_ENABLE=1`	Opts in to iGPU usage (Ollama 0.30+ blocks iGPUs by default)
`OLLAMA_FLASH_ATTENTION=1`	Faster attention math, meaningful speedup on bandwidth-limited iGPUs
`OLLAMA_KEEP_ALIVE=15m`	Keeps models hot in GPU memory between requests

Save (`Ctrl+O`, `Enter`, `Ctrl+X` in nano), then reload:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

Optional — expose Ollama to your LAN. If you want other machines on your network to call this Ollama instance, add a fifth line:

```
Environment="OLLAMA_HOST=0.0.0.0:11434"
```

Skip this if you only need local access. Don't expose Ollama to the internet without a reverse proxy and auth.

Step 7 — Verify everything is wired up

7a — Confirm env vars loaded

```
systemctl show ollama | grep -E "^Environment"
```

You should see all four variables (or five if you added OLLAMA_HOST).

7b — Confirm Ollama detected the GPU

```
sudo journalctl -u ollama -n 50 --no-pager | grep -iE "library|inference compute|dro
```

What success looks like:

```
msg="inference compute" id=0 filter_id=0 library=ROCm compute=gfx1102
name=ROCm0 description="AMD Radeon 780M Graphics" libdirs=ollama,rocm_v7_2
driver=0.0 pci_id=0000:c5:00.0 type=iGPU total="32.0 GiB" available="..."
```

The magic words are `library=ROCm` and `compute=gfx1102`. The "dropping ROCm device" and "dropping integrated GPU" warnings from before should be gone.

7c — Smoke test with a small model

```
ollama pull llama3.2:3b
ollama run llama3.2:3b --verbose "Explain GTT memory in three sentences."
```

In a second terminal during the run:

```
sudo radeontop
```

You should see **Graphics pipe** and **Shader Export** climbing during generation. The `--verbose` flag prints a stats block at the end:

```
eval rate: ~26 tokens/s
```

If you see eval rate ≥ 20 tok/s on a 3B model, you're done. Congratulations.

Performance ceiling

The 780M's memory bandwidth ceiling is unavoidable:

- DDR5-5600 × 2 channels × 8 bytes = **~89 GB/s theoretical**
- ~70-80 GB/s effective in practice
- Tokens-per-second ≈ `bandwidth ÷ model size`

Measured results on this build

Model	Quant	Size	tok/s	Notes
`llama3.2:3b`	Q4	2.0 GB	26.1	Interactive, snappy
`qwen2.5:7b`	Q4	4.4 GB	~18	Daily driver sweet spot
`qwen2.5-coder:7b`	Q4	4.4 GB	~18	Best for code

Model	Quant	Size	tok/s	Notes
`qwen2.5:14b`	Q4	9.0 GB	6.2	Quality > speed, batch jobs
`gpt-oss:20b`	Q4	~12 GB	~4	Slow but works

Interactive chat sweet spot: 7B-class models at 15-20 tok/s.
Quality work where you can wait: 14B at 6 tok/s.
Anything bigger: usable but slow. Don't bother with 70B on this hardware.

Useful follow-on tools

Open WebUI (ChatGPT-style local interface)

```
docker run -d -p 3000:8080 \
  --add-host=host.docker.internal:host-gateway \
  -v open-webui:/app/backend/data \
  --name open-webui \
  --restart always \
  ghcr.io/open-webui/open-webui:main
```

Browse to `http://localhost:3000`. Auto-detects local Ollama.

Live GPU monitoring

```
sudo radeontop          # Live GPU utilization
/opt/rocm/bin/rocm-smi  # ROCm-style GPU status
ollama ps               # Currently loaded models + CPU/GPU split
```

Embedding model for RAG

```
ollama pull nomic-embed-text
```

Pair with a vector DB (Chroma, Qdrant) for local document Q&A.

Troubleshooting

Health check (run this first when something breaks)

```
# 1. GTT allocation still in place?
cat /sys/module/ttm/parameters/pages_limit
# Expect: 8388608

# 2. ROCm still sees GPU?
/opt/rocm/bin/rocminfo | grep gfx1103
# Expect: two lines with gfx1103
```

```
# 3. Ollama env vars still loaded?
systemctl show ollama | grep -E "^Environment"
# Expect: HSA_OVERRIDE_GFX_VERSION=11.0.2 and OLLAMA_IGPU_ENABLE=1 present

# 4. Ollama using ROCm?
sudo journalctl -u ollama -n 50 --no-pager | grep "library="
# Expect: library=ROCm
```

If all four pass and it still won't work, kernel update or Ollama update is the usual culprit.

"dropping ROCm device — no rocblas support for gfx target"

`HSA_OVERRIDE_GFX_VERSION` didn't take effect.

```
systemctl show ollama | grep HSA_OVERRIDE
```

Should show `HSA_OVERRIDE_GFX_VERSION=11.0.2`. If not, re-run `sudo systemctl edit ollama` and check the value (must be `11.0.2`, not `gfx1102` or `1102`).

"dropping integrated GPU"

`OLLAMA_IGPU_ENABLE=1` didn't take effect. Same fix as above, different variable.

`rocminfo` shows no GPU

Check group membership: `groups` should include `render` and `video`. If you just added them, reboot.

If still missing, check that `amdgpu-dkms` did NOT get installed somehow:

```
apt list --installed 2>/dev/null | grep amdgpu-dkms
```

If it's present, remove it: `sudo apt remove --purge amdgpu-dkms && sudo reboot`.

Model loads on CPU instead of GPU

```
ollama ps
```

Look at the `PROCESSOR` column. If it's not `100% GPU`, the model didn't fit. Either:

- Pick a smaller model
- Pick a more aggressive quant (Q3 instead of Q4)
- Verify GTT is at 32GB via Step 2d

Slow performance even on small models

Confirm flash attention is on:

```
systemctl show ollama | grep FLASH_ATTENTION
# Expect: OLLAMA_FLASH_ATTENTION=1
```

Confirm no Vulkan backend conflict:

```
sudo journalctl -u ollama -n 50 --no-pager | grep -i vulkan
```

If Vulkan is being used instead of ROCm, your override didn't apply.

Total reset

```
sudo systemctl stop ollama
sudo apt remove --purge rocm-* hip-* rocminfo
sudo rm /etc/apt/sources.list.d/rocm.list /etc/apt/preferences.d/rocm-pin-600
sudo kernelstub --delete-options "amdgpu.gttsize=32768 ttm.pages_limit=8388608 ttm.pa
sudo reboot
```

Then restart from Step 1.

References

- FastFlowLM repo — what started this whole thing (and why the Pangolin15 can't run it)
- System76 Pangolin tech docs
- Ollama issue #7650 — AMD Radeon 780M on Pop!_OS
- Framework community: ROCm experiments on 780M
- Framework community: unified memory commands
- linux-ng.de — Preparing AMD APUs for LLM usage
- Roderik — Ollama on AMD 8840HS
- AMD ROCm compatibility matrix
- ROCm/ROCm issue #5316 — gfx1103 support discussion

Closing thought

You took a laptop iGPU that AMD's compatibility matrix officially does not support, convinced it to run 14-billion-parameter language models at 70% of theoretical hardware peak, and got there in 90 minutes with one BIOS dead-end, two reboots, and zero kernel module conflicts. That's not a hack — that's just what Linux is for.

The Sega Master System in your laptop is now playing NES cartridges. Class dismissed.